Laura Hietala

Oppimistehtävä 4

Tietojenkäsittelyn koulutus
TKMI23KV

5.12.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

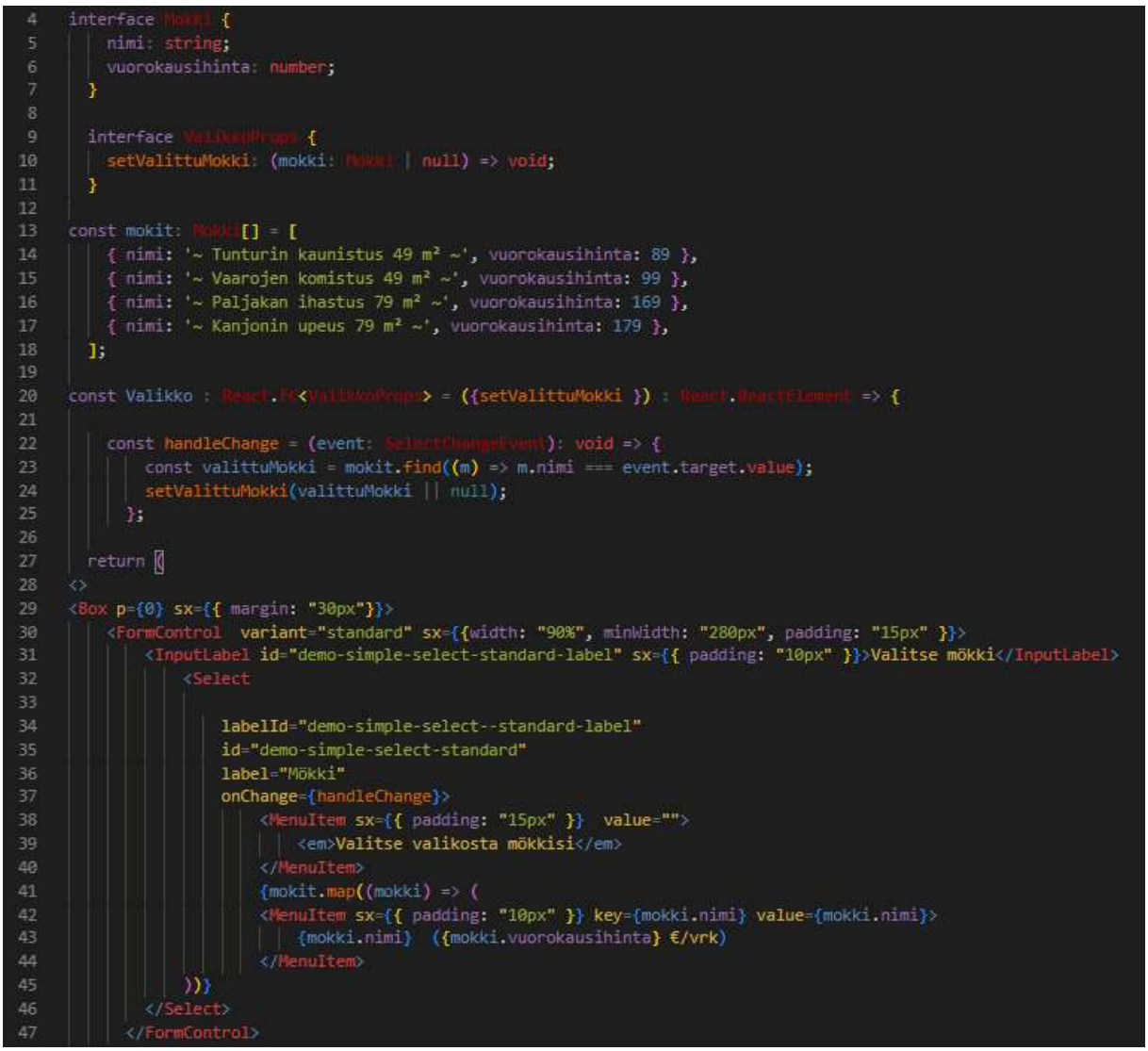
Tehtävänä on toteuttaa React ja MUI -tekniikoin sovellus, jolla voi tehdä varauksen lomamökistä. Toimeksiannossa on vihjeitä/vaatimuksia toteutukseen (sulkumerkeissä kursiivilla). Viittaukset ovat dokumentaatiossa kuvattuja MUI-komponentteja (components).

GitHub*:* *https://github.com/xamk-so1/oppimisteht-v-4-elahi001xamk*

Perusvaatimukset sovellukselle

Vaatimus: Mökki valitaan alasvetovalikosta (*Select*) vähintään neljästä eri vaihtoehdosta. Jokaisella mökillä on eri vuorokausihinta, joka luonnollisesti vaikuttaa varauksen kokonaishintaan. Voit nimetä ja hinnoitella mökit vapaasti, mutta vuorokausihinnan tulee näkyä valinnoissa.

Ratkaisu: Valikko-komponentissa riveillä 13-18 määritellään mökkien array, jossa on 4 eri mökkivaihtoehtoa. Jokaisella mökillä on eri vuorokausihinta (mokki.vuorokausihinta). Alasvetovalikko (Select) on rakennettu FormControl-, InputLabel-, Select-, ja MenuItem-MUI-komponenttien avulla (rivit 30-47). Riveillä 41-45 mapataan mokit-array, ja näin luodaan MenuItem-komponentti jokaiselle mökille. MenuItem näyttää jokaisen mökin nimen ja vuorokausihinnan valikossa.



```
4   interface Mokki {
5       nimi: string;
6       vuorokausihinta: number;
7     }
8
9     interface ValikkoProps {
10      setValittuMokki: (mokki: Mokki | null) => void;
11    }
12
13  const mokit: Mokki[] = [
14      { nimi: '~ Tunturin kaunistus 49 m² ~', vuorokausihinta: 89 },
15      { nimi: '~ Vaarojen komistus 49 m² ~', vuorokausihinta: 99 },
16      { nimi: '~ Paljakan ihastus 79 m² ~', vuorokausihinta: 169 },
17      { nimi: '~ Kanjonin upeus 79 m² ~', vuorokausihinta: 179 },
18    ];
19
20  const Valikko : React.FC<ValikkoProps> = ({setValittuMokki }) : React.ReactElement => {
21
22      const handleChange = (event: SelectChangeEvent): void => {
23          const valittuMokki = mokit.find((m) => m.nimi === event.target.value);
24          setValittuMokki(valittuMokki || null);
25        };
26
27    return (
28  <>
29  <Box p={0} sx={{ margin: "30px"}}>
30      <FormControl  variant="standard" sx={{width: "90%", minWidth: "280px", padding: "15px" }}>
31          <InputLabel id="demo-simple-select-standard-label" sx={{ padding: "10px" }}>Valitse mökki</InputLabel>
32              <Select
33
34                  labelId="demo-simple-select--standard-label"
35                  id="demo-simple-select-standard"
36                  label="Mökki"
37                  onChange={handleChange}>
38                      <MenuItem sx={{ padding: "15px" }}  value="">
39                          <em>Valitse valikosta mökkisi</em>
40                      </MenuItem>
41                      {mokit.map((mokki) => (
42                      <MenuItem sx={{ padding: "10px" }} key={mokki.nimi} value={mokki.nimi}>
43                          {mokki.nimi}  ({mokki.vuorokausihinta} €/vrk)
44                      </MenuItem>
45              ))}
46          </Select>
47        </FormControl>
```

Vaatimus: Varauksen kesto valitaan kokonaisina vuorokausina liukuvalinnasta (*Slider*). Varauksen voi tehdä maksimissaan 14 vuorokaudeksi. Oletuksena on 1 vuorokausi.

Ratkaisu: Liukuvalitsin-komponentissa Slider-komponentti (rivit 57-71) mahdollistaa päivien lukumäärän valinnan. Rivien 68 ja 69 min- ja max- ominaisuudet määrittävät, että päiviä voi varata 1-14. Rivin 66 step-ominaisuus määrittää, että liukuvalitsinta voi liikuttaa aina yhden kokonaisen vuorokauden kerrallaan. Rivin 63 defaultValue asettaa oletuksen 1 vuorokauteen.

```
57      <Slider
58          value={props.paivat}
59          onChange={handleChange}
60          sx={{width: "95%"}}
61          track={false}
62          aria-labelledby="track-false-slider"
63          defaultValue={1}
64          getAriaValueText={valuetext}
65          valueLabelDisplay="auto"
66          step={1}
67          marks ={marks}
68          min={1}
69          max={14}
70          aria-label="Kuinka monta yötä"
71      />
```

Vaatimus: Varaukseen voi liittää myös loppusiivouspalvelun (100 euroa lisää loppusummaan). Valinta tehdään "raksi ruutuun"-valinnalla (*Checkbox*).

Ratkaisu: Tästä huolehtii Loppusiivous-komponentti. Käytän Checbox-komponenttia valintaruutuna (rivi 16). handleCheckboxChange-funktio (määritellään riveillä 9-11) määrittelee, mitä tapahtuu, kun käyttäjä klikkaa valintaruutua. Funktio kutsuu props.setLoppusiivous-funktiota (rivi 5) ja välittää sille tiedon siitä, onko valintaruudussa raksia vai ei (event.target.checked).

```
4   interface LoppusiivousProps {
5     setLoppusiivous: (value: boolean) => void;
6   }
7
8   const Loppusiivous: React.FC<LoppusiivousProps> = (props: LoppusiivousProps) : React.ReactElement => {
9     const handleCheckboxChange = (event: React.ChangeEvent<HTMLInputElement>) => {
10      props.setLoppusiivous(event.target.checked);
11    };
12
13    return (
14      <>
15        <FormControlLabel sx={{marginLeft: "10px"}}
16          control={<Checkbox onChange={handleCheckboxChange} />}
17          label="Loppusiivous 100 €"
18        />
19      </>
20    );
21  }
```

FormControlLabel-komponentti riveillä 15-18 kasaa Checkbox-komponentin ja labelin (rivi 17) samaan kokonaisuuteen. Label kertoo, että loppusiivous maksaa 100 euroa.

Vaatimus: Varauksen kokonaissumma näkyy selkeästi esim. valintojen alapuolella ja se lasketaan aina uudelleen, kun valintoja muutetaan.

Ratkaisu: Tämä tapahtuu Yhteenveto-komponentissa. h2-elementissä rivillä 19 näkyy otsikko "Varauksen yhteenveto". Rivien 20-23 p-elementeissä näytetään yhteenvedon tiedot. Tiedot välitetään ominaisuuksina eri komponenteilta App-pääkomponentin kautta.

```
3  interface Mokki {
4      nimi: string;
5      vuorokausihinta: number;
6    }
7
8  interface YhteenvetoProps {
9    valittuMokki: Mokki | null;
10   paivat: number;
11   loppusiivous: boolean;
12   yhteensa?: number;
13 }
14
15 const Yhteenveto: React.FC<YhteenvetoProps> = ({ valittuMokki, paivat, loppusiivous, yhteensa }) => {
16   return (
17     <div>
18
19       <h2>Varauksen yhteenveto</h2>
20       <p><strong>Mökki:</strong> {valittuMokki ? valittuMokki.nimi : 'Ei valittu'}</p>
21       <p><strong>Varauksen kesto:</strong> {paivat} {paivat === 1 ? 'vuorokausi' : 'vuorokautta'}</p>
22       <p><strong>Loppusiivous:</strong> {loppusiivous ? 'Valittu' : 'Ei valittu'}</p>
23       <p><strong>Yhteensä:</strong> {yhteensa} €</p>
```

Kokonaissumma päivittyy automaattisesti aina, kun Yhteenveto-komponentti saa päivitetyn yhteensa-arvon riville 23. Arvo päivittyy, kun käyttäjä tekee muutoksia valintoihinsa käyttöliittymässä. Näitä tilatietoja hallinnoidaan Laskuri-komponentissa updateYhteensa-funktion avulla (rivit 25-34). Se päivittää yhteensa-tilaa (määritellään rivillä 19 ja päivitetään rivillä 33) aina, kun valinnat muuttuvat. Laskurin useEffect (rivit 21-23) aiheuttaa sen, että updateYhteensa-funktiota kutsutaan aina, kun sen tilat muuttuvat. updateYhteensa-funktio välittää päivitetyn yhteensa-tilan Yhteenveto-komponentille propsina.

```tsx
interface LaskuriProps {
    children?: React.ReactElement | React.ReactElement[] | string | number;
  }

  const Laskuri: React.FC<LaskuriProps> = (props: LaskuriProps): React.ReactElement => {
    const [paivat, setPaivat] = useState<number>(1);
    const [valittuMokki, setValittuMokki] = useState<Mokki | null>(null);
    const [loppusiivous, setLoppusiivous] = useState<boolean>(false);
    const [yhteensa, setYhteensa] = useState<number>(0);

    useEffect(() => {
      updateYhteensa();
    }, [valittuMokki, paivat, loppusiivous]);

  const updateYhteensa = () : void => {
    let summa = 0;
    if (valittuMokki) {
      summa += paivat * valittuMokki.vuorokausihinta;
    }
    if (loppusiivous) {
      summa += 100;
    }
    setYhteensa(summa);
  };

      const handleValittuMokkiChange = (mokki: Mokki | null) => {
        setValittuMokki(mokki);
      };

      const handlePaivatChange = (value: number) => {
    setPaivat(value);
  };

      return (
        <>
          <Valikko setValittuMokki={handleValittuMokkiChange} />
          <Liukuvalitsin paivat={paivat} setPaivat={handlePaivatChange}  />
          <Yhteenveto valittuMokki={valittuMokki} paivat={paivat} loppusiivous={loppusiivous} yhteensa={yhteensa} />
        </>
```

laskeYhteensa-funktio (määritellään App-komponentissa riveillä 13-19) laskee kokonaissumman ja sitä kutsutaan aina, kun updateYhteensa-funktio päivittää yhteensa-tilaa. laskeYhteensa-funktiota kutsutaan App-komponentissa rivillä 31.

```tsx
const App: React.FC = (): React.ReactElement => {
  const [valittuMokki, setValittuMokki] = useState<{ nimi: string; vuorokausihinta: number } | null>(null);
  const [paivat, setPaivat] = useState<number>(1);
  const [loppusiivous, setLoppusiivous] = useState<boolean>(false);

  const laskeYhteensa = (): number => {
    let summa = valittuMokki ? paivat * valittuMokki.vuorokausihinta : 0;
    if (loppusiivous) {
      summa += 100;
    }
    return summa;
  };

  return (
    <>
      <Sivu>

        <Valikko setValittuMokki={setValittuMokki} />

        <Liukuvalitsin paivat={paivat} setPaivat={setPaivat} />

        <Loppusiivous setLoppusiivous={setLoppusiivous} />

        <Yhteenveto valittuMokki={valittuMokki} paivat={paivat} loppusiivous={loppusiivous} yhteensa={laskeYhteensa()} />
```

Lisävaatimukset sovellukselle

En toteuttanut lisävaatimuksia.